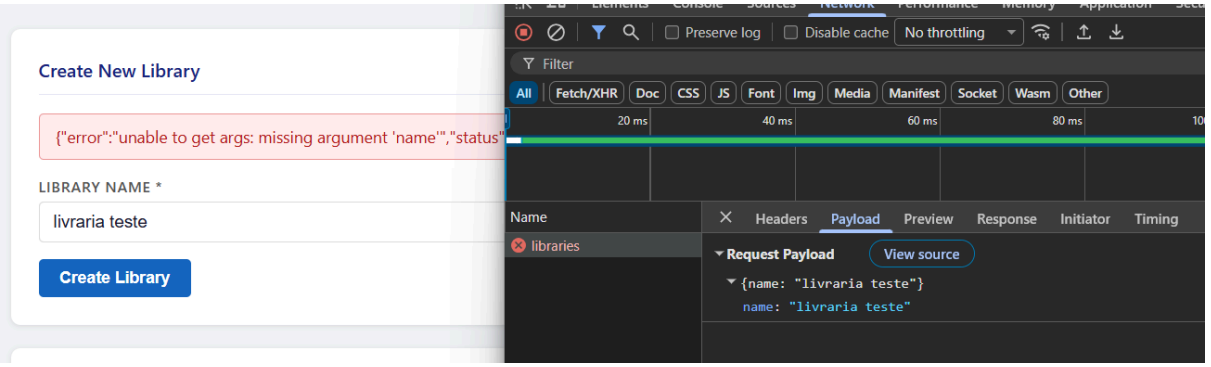
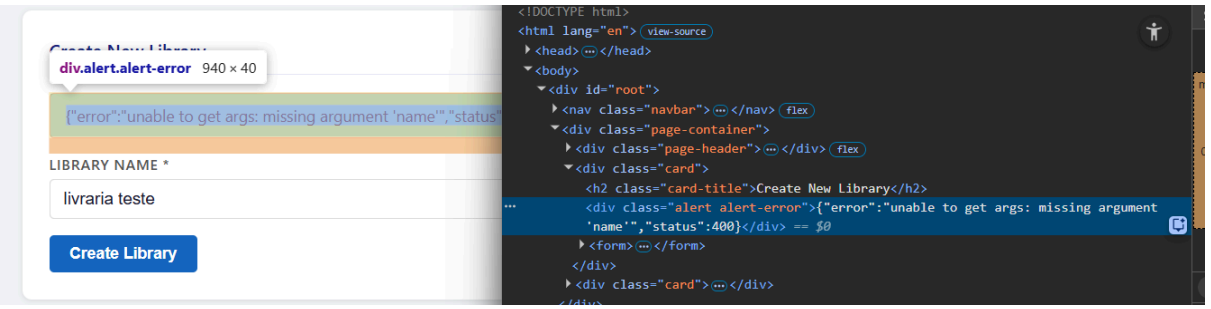
Relatório de Incidente de Integração (API Contract Mismatch) - Goledger Challenge

ID do Chamado: BUG-008 | **Prioridade:** Alta (Blocker de Cadastro) **Módulo:** Cadastro de Livrarias / Unidades (Front-end ↔ API) **Plataforma:** Aplicação Web (React/Vite) **Relator:** Vitória (QA)




div.alert.alert-error 940 × 40
{"error":"unable to get args: missing argument 'name'","status"
LIBRARY NAME *
livraria teste
Create Library
<!DOCTYPE html>
<html lang="en">
<head></head>
<body>

<nav class="navbar">

<h2 class="card-title">Create New Library</h2>
{"error":"unable to get args: missing argument 'name'","status":400} == $0
<form></form>

divergência de nomenclatura no contrato de dados, onde o Front-end envia um parâmetro que o Back-end não reconhece, impedindo a persistência da nova unidade.

2. Passos para Reproduzir (Reproducibility)

- **Ação:** Acesse o formulário de "Nova Livraria".
- **Execução:** Preencha os campos obrigatórios e clique em "Salvar".
- **Análise Técnica:** Abra o console de desenvolvedor (F12) e verifique a aba *Network*.
- **Constatação:** Observe que a requisição POST para o endpoint de livrarias é rejeitada pelo servidor com o código 400, devido ao erro de mapeamento no payload.

3. Análise da Causa Raiz (Divergência de Atributos) Através da análise técnica do tráfego de dados e do código de integração, confirmou-se que o erro de integração reside no nome da propriedade enviada. O Front-end está disparando a requisição utilizando a chave `libraryName`, enquanto o contrato atualizado da API espera receber o atributo simplificado como `name`. Por não encontrar o campo obrigatório `name`, o Back-end rejeita o pedido como inválido (Bad Request).

antes

```go
func CreateLibrary(c *gin.Context) {
    var req libraryRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    params := map[string]interface{}{
        "libraryName": req.Name,
    }
```

depois

```go
// CreateLibrary creates a new library via the org3 CCAPI.
func CreateLibrary(c *gin.Context) {
    var req libraryRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    params := map[string]interface{}{
        "name": req.Name,
    }
```

4. Impacto Operacional

- **Impedimento Funcional:** Bloqueio total da funcionalidade de gestão de livrarias.
- **Manutenibilidade:** Demonstra a necessidade de atualização dos modelos de dados no Front-end para refletir as mudanças realizadas no Schema da API.

5. Recomendações Técnicas e Solução

- **Refatoração de Payload:** É necessário alterar a chave de envio no arquivo de serviço ou no componente de formulário de livrarias.
- **Ação:** Substituir o termo `libraryName` por `name` em todos os objetos de transporte de dados (DTOs) que se comunicam com o endpoint de criação de livrarias.

6. Evidência de Fechamento Após a correção da nomenclatura do atributo, o sistema deve processar o cadastro com sucesso, retornando o status `201 Created` e refletindo a nova livraria na listagem global.